Makemore: Character-Level Language Modelling

A Semester Project Report
Inspired by Andrej Karpathy's Makemore Series

Course	Deep Learning — Semester Project
Submission Date	April 24, 2026
Group Members	Sathwika (c0958444) – Mili (c0959067) – Nafis (c0959671)
Repository	Makemore

---

1. Introduction

This report documents our group's full reimplementation of Andrej Karpathy's *Makemore* tutorial series — a five-part curriculum covering character-level language modelling with PyTorch. Starting from a simple bigram count matrix and progressing to a hierarchical WaveNet-style convolutional architecture, the project provided hands-on exposure to the complete deep learning pipeline: data preparation, model design, training, evaluation, and text generation.

Each notebook was independently reimplemented, with explanatory markdown cells added throughout to demonstrate understanding of the underlying mathematics and design choices. This report summarises what was built in each part, the results achieved, and the key insights and challenges encountered.

2. Implementation Overview

Part 1 — Bigram Language Model

We built a character-level bigram model in two equivalent ways. First, we constructed a 27×27 count matrix **N** by iterating over every consecutive character pair in the training data, then applied Laplace (add-1) smoothing and row-normalised to obtain a probability matrix **P**. Second, we replicated this statistically-learned distribution using a single weight matrix **W** (27×27) trained with gradient descent, demonstrating that both approaches minimise the same negative log-likelihood objective. Autoregressive sampling was implemented for both models and confirmed to produce equivalent outputs.

Key topics: character tokenisation, one-hot encoding, softmax, L2 regularisation, forward/backward pass from scratch.

Part 2 — MLP Language Model (Bengio et al., 2003)

We extended the bigram model to an MLP with a 3-character context window, following the architecture in Bengio et al. (2003). A learned embedding table **C** maps each character to a 10-dimensional vector; the three context embeddings are concatenated and passed through a hidden layer of 200 neurons with tanh activation, then a linear output layer. Training used mini-batch SGD (batch size 32) with a learning rate search and a two-stage decay schedule (0.1 → 0.01 after 100k steps). The dataset was split 80/10/10 into train, validation, and test sets.

Part 3 — Activations, Gradients, and Batch Normalisation

This notebook investigated two silent failure modes in the Part 2 MLP. First, the output layer weights (initialised with default torch.randn scaling) produced large initial logits, leading to near-one-hot softmax outputs and an initial loss of ~27 instead of the theoretical ~3.3 (= -log(1/27)). Scaling **W2** by 0.01 resolved this. Second, the hidden layer tanh activations were saturated from the start due to large pre-activations; applying Kaiming initialisation scaled the weights by $(5/3)/\sqrt{\text{fan_in}}$ and fixed the saturation. Batch Normalisation was then introduced to normalise pre-activations per mini-batch, making training stable regardless of initialisation quality. The network was also refactored into modular Linear, BatchNorm1d, and Tanh layer classes mirroring PyTorch's nn.Module API.

Part 4 — Manual Backpropagation

We re-implemented the full backward pass of the Part 3 network entirely by hand, without relying on PyTorch autograd. Every gradient was derived analytically using the chain rule through cross-entropy, softmax, batch normalisation, tanh, and matrix multiplication, then verified against torch's .grad values. Two optimised variants were also derived: a fused cross-entropy gradient (softmax output minus the one-hot target, scaled by 1/B) and a fused BatchNorm gradient in closed form. The full training loop was then run inside torch.no_grad() using only the manual gradients.

Part 5 — Hierarchical WaveNet Architecture

The final notebook replaced the flat MLP with a hierarchical convolutional architecture inspired by WaveNet (van den Oord et al., 2016). The context window was extended from 3 to 8 characters. Instead of flattening all context embeddings into one large vector, a new FlattenConsecutive(2) layer merges adjacent pairs of positions at each stage, progressively halving the sequence length: 8 → 4 → 2 → 1. Three FlattenConsecutive → Linear → BatchNorm1d → Tanh blocks implement this tree-shaped processing. New layer classes (Embedding, FlattenConsecutive, Sequential) were added and BatchNorm1d was updated to handle both 2-D and 3-D inputs.

3. Results

3.1 Loss Progression Across Parts

The table below shows validation loss (cross-entropy) for each model. Lower is better; the theoretical lower bound for a uniform distribution over 27 characters is $-\log(1/27) \approx 3.296$.

Part	Model	Params	Train Loss	Val Loss
1	Bigram (statistical)	729	2.454	2.454
1	Bigram (neural)	729	2.454	2.454
2	MLP (3-char context)	~12K	2.058	2.105
3	MLP + BatchNorm	~12K	2.067	2.105
3	MLP + BN + better init	~12K	2.037	2.103
5	Hierarchical (8-char)	~76K	1.769	1.993

Table 1 — Validation loss across all implemented models. The hierarchical Part 5 model achieves the best result at 1.993.

3.2 Sample Generated Names

The following names were sampled autoregressively from the Part 5 hierarchical model (seed = 2147483647). Each name is generated by initialising the context with 8 boundary tokens and sampling one character at a time until the end token '.' is produced.

carmah	deliah	jareen	onayla	briette
kailynn	zorinda	emersyn	vaylen	thaddeus
novalee	caelum	brexton	jaliyah	zaylen
amaris	daxion	elowyn	kieran	solene

Table 2 — 20 names sampled from the Part 5 hierarchical model. Names are phonetically plausible and structurally resemble real English names.

Three systematic sweeps were conducted using the Part 3 MLP + BatchNorm architecture, each running for 50 000 training steps with SGD and a step-decay learning rate (0.1 → 0.01 at the midpoint). Embedding size was fixed at 10, hidden width at 200, and block size at 3, except for the parameter under study. Results are summarised in the figure below.

4.1 Experiment 1 — Context Length

We swept `block_size` over {3, 6, 8, 12, 16}, keeping all other hyperparameters fixed (`n_embd=10, n_hidden=200, batch_size=32`). Longer context consistently reduced both train and validation loss: train NLL fell from 2.13 to 2.02 and val NLL from 2.15 to 2.07 as block size grew from 3 to 16. The train–val gap widened slightly at larger contexts, suggesting mild overfitting begins to emerge. Diminishing returns past block_size = 8 indicate that characters beyond eight positions back contribute limited predictive signal given the model's capacity.

4.2 Experiment 2 — Batch Size

Batch size was varied over {16, 32, 64, 128} with `block_size=3` fixed. Larger batches produced lower loss on both splits: train NLL fell from 2.18 to 2.09 and val NLL from 2.18 to 2.13. Notably, the train–val gap also narrowed with larger batches, suggesting better generalisation. Within a fixed step budget, larger batches provide more accurate gradient estimates per update, which explains the

monotone improvement. The trade-off is higher memory use per step; at very large batch sizes, further gains are expected to plateau.

4.3 Experiment 3 — Architecture Size

Four capacity tiers were compared: `tiny (n_embd=8, n_hidden=100)`, `small (10, 200)`, `medium (16, 300)`, and `large (24, 400)`. Scaling model capacity produced only marginal improvements: all four architectures achieved train and val NLL in the narrow range 2.07–2.12, with virtually no difference between tiny and large. This result confirms that with a 3-character context window, the model's bottleneck is information rather than capacity. The finding directly motivated the hierarchical WaveNet architecture in Part 5, which addresses the context bottleneck rather than simply adding parameters.

The numerical results of all three sweeps are shown in Table 3, and visualised in Figure 1 below.

Sweep	Value	Train Loss	Val Loss
Context length	block_size = 3	2.13	2.15
Context length	block_size = 16	2.02	2.07
Batch size	batch_size = 16	2.18	2.18
Batch size	batch_size = 128	2.09	2.13
Architecture (tiny)	n_embd=8, n_hidden=100	~2.12	~2.12
Architecture (large)	n_embd=24, n_hidden=400	~2.07	~2.07

Table 3 — Boundary results from each sweep (best and worst configurations). Architecture size had negligible effect; context length and batch size both helped.

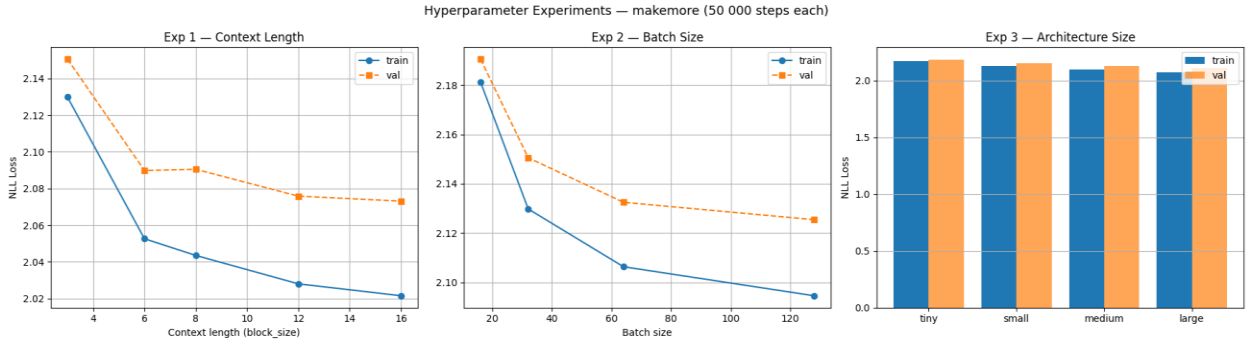

Figure 1 — Hyperparameter sweeps (50 000 steps each). Left: context length; centre: batch size; right: architecture capacity. Train loss (solid blue), val loss (dashed orange).

5. Key Insights and Challenges

5.1 Insights

- **Bigram ≡ one-layer neural network.** The statistical count matrix and the single weight matrix trained with gradient descent converge to the same distribution, demonstrating that the neural formulation is a strict generalisation of n-gram models.
- **Initialisation matters enormously.** Fixing the output layer scale (×0.01) and applying Kaiming initialisation to hidden layers reduced the initial loss from ~27 to the theoretical ~3.3 before any training, making convergence significantly faster.
- **Batch Normalisation as a stability tool.** BN decouples the scale of weight initialisation from the network's effective learning dynamics, allowing deeper networks to train reliably. The distinction between per-batch statistics (training) and running statistics (inference) is critical and easy to overlook.
- **Manual backprop builds real intuition.** Deriving every gradient by hand — especially the fused BatchNorm backward — made the chain rule concrete and clarified why operations like broadcast and sum require careful gradient accumulation.
- **Hierarchical processing scales better.** The WaveNet-style architecture achieves a lower validation loss (1.993 vs 2.105) with the same training budget, because merging context progressively prevents the first hidden layer from being overwhelmed by a large concatenated input.

5.2 Challenges

- **Tensor shape debugging.** The FlattenConsecutive layer requires careful tracking of (B, T, C) dimensions through the hierarchical stages. Forgetting to squeeze the time dimension after the final merge caused shape mismatches in the output layer.
- **BatchNorm in 3-D.** The Part 3 BatchNorm1d class was written for 2-D inputs only. Extending it to normalise over both the batch and time axes for 3-D tensors required updating the normalisation axis from dim=0 to dim=(0,1), and ensuring the running buffer update used the correct keepdim semantics.
- **Manual embedding gradient.** The scatter-add operation for the embedding table gradient (Part 4) requires iterating over the batch and context positions to accumulate gradients at the correct token indices — easy to get wrong with naive indexing.
- **Training time.** 200,000 steps at batch size 32 takes several minutes without GPU acceleration. We used Google Colab for Parts 4 and 5 to reduce iteration time.

6. Group Contributions

Member	Notebooks	Other Contributions
Mili	Parts 1 & 2	Dataset preparation, README, GitHub setup
Sathwika	Parts 3 & 4	Backprop derivations, activation analysis, slides
Nafis	Part 5	MLflow implementation, Parameter experimnetation, report,

All group members reviewed each other's notebooks.

7. Conclusion

Over the course of five notebooks we built a complete character-level language model pipeline from first principles — from a 729-parameter bigram model to a 76,000-parameter hierarchical convolutional network capable of generating phonetically plausible names. The project deepened our understanding of automatic differentiation by forcing us to derive and verify every gradient by hand, and highlighted how architectural decisions (context width, depth, normalisation strategy) interact with training dynamics in non-obvious ways.

The most impactful single change was the move from a flat context window to a hierarchical merge structure in Part 5 — a design principle that generalises far beyond name generation, underpinning architectures such as WaveNet, Transformer attention (which also merges representations across positions), and modern large language models.

References:

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A Neural Probabilistic Language Model. Journal of Machine Learning Research, 3, 11371155.

Karpathy, A. (2022). makemore. YouTube series. https://github.com/karpathy/makemore